

Diverse Fault Attacks on Dilithium and Variant Implementation: Skipping, Aborting and Zeroing

Hao Yuan¹, Yuejun Liu^{1✉}, Jingdian Ming¹, Yongbin Zhou^{1,2✉}

¹School of Cyber Science and Engineering, Nanjing University of Science and Technology, Nanjing, China

²Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

{y_yuanhao, liuyuejun, mingjingdian, zhouyongbin}@njust.edu.cn

Abstract—As one of the first post-quantum digital signature schemes standardized by NIST, Dilithium has gained significant attention due to its potential role in securing communication in the quantum era. However, ensuring the security of its practical implementations, particularly against fault attacks, remains a significant challenge. Despite various proposed countermeasures, their effectiveness remains inadequately explored. This paper presents a comprehensive analysis of fault attacks on Dilithium, introducing five distinct attack techniques across three fault types: instruction-skipping, loop-aborting, and zeroing, which target the polynomial structure and Number Theoretic Transform (NTT) operations in Dilithium. These attacks are shown to significantly reduce the number of faulty signatures required for private key recovery, with the most efficient technique requiring as few as one faulty signature. Our methods successfully compromise both the reference and the protected implementation designed to resist skip-addition attacks. Experimental validation on an ARM Cortex-M4 platform demonstrates the effectiveness of these attacks, with success rates ranging from 29% to 63%, enabling full private key recovery in both deterministic and randomized Dilithium implementations. These findings underscore the need for more robust, fault-resilient post-quantum cryptographic implementations.

Index Terms—Post-Quantum Cryptography, Dilithium, Fault Attack, Number Theoretic Transform, Embedded Security

I. INTRODUCTION

The advent of quantum computing poses a significant threat to classical public-key cryptography, as quantum algorithms like Shor's can efficiently solve problems such as integer factorization and discrete logarithms in polynomial time [1]. This has driven the development of post-quantum cryptography (PQC), with the National Institute of Standards and Technology (NIST) leading the effort to standardize secure alternatives. Among the first post-quantum standards, Dilithium (now ML-DSA) [2], a lattice-based digital signature algorithm, is based on the hardness of the Modular Learning With Errors (MLWE) and Modular Short Integer Solution (MSIS) problems, which are widely believed to remain intractable even against quantum adversaries.

Although Dilithium is provably theoretically secure, practical implementations are vulnerable to physical attacks [3], including side-channel attacks (SCAs) [4] and fault attacks [5]. Compared to SCAs, fault attacks are more devastating, actively inducing errors to produce faulty outputs that directly reveal sensitive data. With PQC schemes like Dilithium increasingly deployed in embedded devices, ensuring their resilience against fault attacks is crucial for real-world security.

Recent studies have explored fault attacks on Dilithium target various components of its signing process, categorized into three types: skip-addition attacks [6]–[8], which skip masking polynomial additions to expose secret data; loop-abort attacks [9], [10], which halt loops early to induce zero coefficients for private key recovery via lattice reduction or integer linear programming; and NTT-specific attacks [11], which target twiddle factors in number-theoretic transform (NTT) to collapse them to zero.

Despite these efforts, significant gaps remain. Most attacks focus on the reference and *pqm4* implementations [12], [13], as well as the deterministic Dilithium [14], leaving randomized versions, designed to enhance fault resistance, underexplored. Additionally, existing attacks often require high fault precision, numerous faulty signatures, or rely on challenging assumptions, limiting their practicality. Systematic evaluations across fault models, implementations, and variants are also lacking. Furthermore, systematic evaluations across different fault models and implementations are still lacking.

This work addresses these gaps by analyzing fault vulnerabilities in both deterministic and randomized Dilithium, targeting the reference implementation [12] and a variant implementation [13] designed to resist skip-addition attacks. We introduce novel fault models for polynomial addition and NTT operations in the signing algorithm, achieving key recovery with fewer faulty signatures. Our contributions are summarized as follows:

- We propose two **function-call skipping attacks** on addition subroutines, applicable to both the reference and the optimized *NTT-protected* implementations. These attacks enable recovery of the private key from as few as one faulty signature—down from the $\ell \cdot n$ required by previous approaches of this kind [7], [8] ($\ell \cdot n \geq 1024$).
- We propose two **loop-abort fault attacks** targeting polynomial-level and coefficient-level additions. Importantly, the polynomial-level attack is applicable to *NTT-protected* implementations as well. Unlike previous loop-abort techniques [9], [10], which often suffer from noise and shuffling, our methods yield clean, exploitable equations, enabling recovery of the private key s_1 with fewer faulty signatures.
- We propose a **single-twiddle-factor zeroing attack** on the *NTT-protected* implementation, achieved by skipping a single NTT assignment. This attack requires only two

TABLE I: Comparison of Fault Attacks on ML-DSA/Dilithium

Fault Type	Attack Scheme	Fault Target	Impl.*	Faulty Signatures Required		
				II	III	V
Skipping	Skip Addition Attack [7]	$z = y + c \cdot s_1$	■/▲	1024	1280	1792
	Skip Fault Correction [8]			1024	1280	1792
Aborting	Loop Abort Fault [9]	$y \leftarrow \text{ExpandMask}$	■	4	5	7
Random Fault	NTT_Twiddle_Fault [11]	NTT(y)	●	4	5	7
Skipping	[†] Polyvec-Level Addition Skip	$z = y + c \cdot s_1$	■/●	1	1	1
	[†] Poly-Level Addition Skip			4	5	7
Aborting	[†] Poly-Level Loop Abort*		■/●	2	2	2
	[†] Coeff-Level Loop Abort			4	5	7
Zeroing	[†] Single-Twiddle-Factor Zeroing	NTT(y)	●	8	10	14

* **Impl.:** ■ denotes the official reference implementation or *pqm4* implementation, ● indicates the NTT-protected implementation described in (II.1), and ▲ represents the implementation that reorders the additions in the computation of z .

* **Poly-Level Loop Abort:** Under this attack, the first polynomial of s_1 is unrecoverable; therefore, one additional signature is required for it, while a single signature suffices to recover all of the remaining $\ell-1$ polynomials.

[†] Denotes attack techniques proposed in this paper.

faulty signatures to recover a single polynomial from s_1 , relaxing the all-twiddle-zeroing requirement of [11] and making the attack more feasible in practice.

- Through experiments on an ARM Cortex-M4 platform using ChipWhisperer-Lite, we validate the practicality of all proposed attacks, achieving success rates ranging from 29% to 63%, enabling full recovery of the private key s_1 in both deterministic and randomized Dilithium implementations.

Our attacks, compared with prior work in Table I, offer greater efficiency and broader applicability, highlighting the need for robust countermeasures in Dilithium deployments.

II. BACKGROUND

A. Notation

We denote the ring of integers modulo q as $\mathbb{Z}_q$, and define the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Elements in this ring are denoted using regular lowercase letters (e.g., $f \in R_q$). The coefficient of x^i in a polynomial f is written as f_i . Bold lowercase letters (e.g., $\mathbf{f} \in R_q^k$) represent vectors of polynomials, with $\mathbf{f}[i]$ denoting the i -th polynomial in the vector. Furthermore, $\mathbf{f}[i][j]$ denotes the coefficient of x^j in the i -th polynomial. Bold uppercase letters (e.g., $\mathbf{F} \in R_q^{k \times \ell}$) represent matrices over R_q .

In Dilithium, all polynomial multiplications are performed in the NTT domain, where a single polynomial is the basic computational unit. Given two polynomials $a, b \in R_q$, their product is computed as: $a \cdot b = \text{NTT}^{-1}(\text{NTT}(a) \circ \text{NTT}(b))$, where $\circ$ denotes coefficient-wise (pointwise) multiplication. For a vector of polynomials $\mathbf{v} \in R_q^k$, we define $\hat{\mathbf{v}} = \text{NTT}(\mathbf{v})$ to denote component-wise NTT representation. Multiplication of a polynomial $a \in R_q$ with a polynomial vector $\mathbf{v} \in R_q^k$ is defined as multiplying a with each component polynomial of $\mathbf{v}$. In this paper, if a fault occurs in a variable such as a , we use a' to denote its faulted version.

B. Dilithium

Dilithium operates over the polynomial ring R_q , with fixed parameters $n = 256$ and $q = 2^{23} - 2^{13} + 1$. These choices enable efficient polynomial arithmetic using the Number Theoretic Transform (NTT). The scheme supports flexible module dimensions (k, ℓ) , corresponding to the three NIST PQC security levels for Dilithium; specifically, Levels II, III, and V use $(k, \ell) = (4, 4)$, $(6, 5)$, and $(8, 7)$, respectively.

Dilithium provides two signing versions: *deterministic* and *randomized*. The key distinction lies in the generation of the internal randomness used during the signing process. Specifically, the randomized version generates a fresh random value rnd at line 4 of the signing algorithm II.2 to derive a unique per-signature seed. In contrast, the deterministic version sets rnd to a fixed zero value, resulting in identical signatures when signing the same message multiple times.

The Dilithium signature scheme consists of three fundamental procedures: *key generation*, *signing*, and *verification*, which are detailed as follows.

Key Generation (Algorithm II.1): The key generation begins by expanding a uniformly random seed using

Algorithm II.1 Dilithium.KeyGen()

Output: Public key pk and private key sk

- 1: $\xi \leftarrow \{0, 1\}^{256}$
 - 2: $(\rho, \rho', K) \leftarrow \{0, 1\}^{256} \times \{0, 1\}^{512} \times \{0, 1\}^{256} \leftarrow H(\xi \parallel k \parallel \ell, 1024)$
 - 3: $\hat{\mathbf{A}} \in R_q^{k \times \ell} \leftarrow \text{ExpandA}(\rho)$
 - 4: $(s_1, s_2) \leftarrow \text{ExpandS}(\rho')$
 - 5: $\mathbf{t} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(s_1)) + s_2$
 - 6: $(\mathbf{t}_1, \mathbf{t}_0) \leftarrow \text{Power2Round}(\mathbf{t})$
 - 7: $pk \leftarrow \text{pkEncode}(\rho, \mathbf{t}_1)$
 - 8: $tr \leftarrow H(pk, 512)$
 - 9: $sk \leftarrow \text{skEncode}(\rho, K, tr, s_1, s_2, \mathbf{t}_0)$
 - 10: **return** (pk, sk)
-

SHAKE-256, which is used to derive both the public matrix $\mathbf{A} \in R_q^{k \times \ell}$ and the private key vector $\mathbf{s}_1 \in R_q^\ell$ and $\mathbf{s}_2 \in R_q^k$. The public component is computed as $\mathbf{t} = \mathbf{A} \cdot \mathbf{s}_1 + \mathbf{s}_2$. To reduce the size of the public key, the vector $\mathbf{t}$ is decomposed into high and low bits: the most significant d bits $\mathbf{t}_1$ are retained, while the least significant bits $\mathbf{t}_0$ are discarded. The public key consists of $(\rho, \mathbf{t}_1)$, where ρ is the seed used to expand $\mathbf{A}$, and the private key includes $(\rho, K, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$, where K is a secret seed for signing and $\text{tr} = H(\text{pk})$ is the hash of the public key.

Signing (Algorithm II.2): To sign a message M , the signer first computes the message digest $\mu = H(\text{tr} \parallel M)$. Then, a per-signature seed is derived as $\rho' = H(K \parallel \text{rnd} \parallel \mu)$, where rnd is either a random value (*randomized* version) or a zero vector (*deterministic* version). From this seed, a mask vector

Algorithm II.2 Dilithium $\text{sign}(sk, M)$

Input: message M , $sk = (\rho, K, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$

Output: σ

```

1:  $\hat{\mathbf{A}} \in R_q^{k \times \ell} \leftarrow \text{ExpandA}(\rho)$ 
2:  $\mu \in \{0, 1\}^{512} \leftarrow H(\text{tr} \parallel M, 512)$ 
3:  $\kappa := 0, (\mathbf{z}, \mathbf{h}) \leftarrow \perp$ 
4:  $\text{rnd} \leftarrow \{0, 1\}^{256}$  (or  $\text{rnd} \leftarrow \{0\}^{256}$  for the optional
   deterministic version)
5:  $\rho' \in \{0, 1\}^{512} \leftarrow H(K \parallel \text{rnd} \parallel \mu, 512)$ 
6: while  $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$  do
7:    $\mathbf{y} \in S_{\gamma_1}^\ell \leftarrow \text{ExpandMask}(\rho', \kappa)$ 
8:    $\mathbf{w} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{y}))$ 
9:    $\mathbf{w}_1 \leftarrow \text{HighBits}(\mathbf{w})$ 
10:   $\tilde{c} \in \{0, 1\}^{256} \leftarrow H(\mu \parallel \text{w1Encode}(\mathbf{w}_1), \lambda/4)$ 
11:   $c \in R_q \leftarrow \text{SampleInBall}(\tilde{c})$ 
12:   $\hat{c} \leftarrow \text{NTT}(c)$ 
13:   $\mathbf{z} \leftarrow \mathbf{y} + \text{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_1)$ 
14:   $\mathbf{r}_0 \leftarrow \text{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)$ 
15:  if  $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$  or  $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$  then
16:     $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$ 
17:  else
18:     $\mathbf{h} \leftarrow \text{MakeHint}_q(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$ 
19:    if  $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$  or  $|\mathbf{h}|_{h_j=1} > \omega$  then  $(\mathbf{z}, \mathbf{h}) := \perp$ 
20: return  $\sigma \leftarrow \text{sigEncode}(\mathbf{z} \bmod^\pm q, \mathbf{h}, \tilde{c})$ 
```

Algorithm II.3 Dilithium $\text{verify}(pk, M, \sigma)$

Input: Public key pk , message M , signature σ

```

1:  $(\rho, \mathbf{t}_1) \leftarrow \text{pkDecode}(\text{pk})$ 
2:  $(\mathbf{z}, \mathbf{h}, \tilde{c}) \leftarrow \text{sigDecode}(\sigma)$ 
3:  $\hat{\mathbf{A}} \in R_q^{k \times \ell} \leftarrow \text{ExpandA}(\rho)$ 
4:  $\mu \in \{0, 1\}^{512} \leftarrow H(H(\text{pk}, 512) \parallel M, 512)$ 
5:  $c \in R_q \leftarrow \text{SampleInBall}(\tilde{c})$ 
6:  $\mathbf{w}'_{\text{Approx}} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{z}) - \hat{c} \circ \text{NTT}(\mathbf{t}_1 \cdot 2^d))$ 
7:  $\mathbf{w}'_1 \leftarrow \text{UseHint}_q(\mathbf{h}, \mathbf{w}'_{\text{Approx}}, 2\gamma_2)$ 
8:  $\tilde{c}' \leftarrow H(\mu \parallel \text{w1Encode}(\mathbf{w}'_1), \lambda/4)$ 
9: if  $\tilde{c} = \tilde{c}'$  and  $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ 
   and  $\# \text{ of 1's in } \mathbf{h} \leq \omega$  then
10:   accept
```

$\mathbf{y}$ is sampled with coefficients drawn uniformly at random from the integer interval $[-\gamma_1, \gamma_1]$ (with $\gamma_1 \in \{2^{17}, 2^{19}\}$); it is then transformed into the NTT domain and multiplied with $\hat{\mathbf{A}}$ to compute the commitment $\mathbf{w}_1$. The challenge $c \in R_q$ is then sampled by hashing $\mu \parallel \mathbf{w}_1$, and the response vector is computed as $\mathbf{z} = \mathbf{y} + \text{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_1)$. A rejection check is performed to ensure $\mathbf{z}$ and a derived vector $\mathbf{r}_0$ satisfy the norm bounds. If accepted, a hint vector $\mathbf{h}$ is generated to help the verifier reconstruct $\mathbf{w}_1$. The final signature is $\sigma = (\mathbf{z}, \mathbf{h}, \tilde{c})$.

Verification (Algorithm II.3): To verify a signature $\sigma = (\mathbf{z}, \mathbf{h}, \tilde{c})$, the verifier first reconstructs the challenge c from $\tilde{c}$, and computes the commitment $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t}_1 \cdot 2^d)$. Then, the verifier recomputes μ and checks whether $\tilde{c} = H(\mu \parallel \mathbf{w}'_1)$, while also ensuring that $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ and the number of nonzero entries in $\mathbf{h}$ does not exceed ω . If all checks pass, the signature is accepted as valid.

C. NTT-protected Implementation

To counter addition-skipping fault attacks [7], [8], a defensive strategy was first proposed in [7]. Specifically, the standard implementation $\mathbf{z} = \mathbf{y} + \text{NTT}^{-1}(\text{NTT}(c) \circ \text{NTT}(\mathbf{s}_1))$ (Algorithm II.2, Line 13) is replaced with the following alternative:

$$\mathbf{z} = \text{NTT}^{-1}(\text{NTT}(\mathbf{y}) + \text{NTT}(c) \circ \text{NTT}(\mathbf{s}_1)) \quad (\text{II.1})$$

In this implementation, all additions are performed in the NTT domain. If an addition-skipping behavior occurs in this domain, the resulting error propagates across all coefficients due to the $\log_2 n$ -stage butterfly network structure of the NTT, as shown in Fig. 1. Such skipped addition behavior is effectively equivalent to zeroing out one coefficient in $\hat{\mathbf{y}}$, resulting in a modified polynomial denoted by $\hat{\mathbf{y}}'$. The corresponding inverse transform, $y' = \text{NTT}^{-1}(\hat{\mathbf{y}}')$, is very likely to contain coefficients exceeding the rejection bound γ_1 , causing rejection (Algorithm II.2, Line 15), with a passing probability of about 2^{-4320} [7]. As a result, this implementation is resistant to addition-skipping fault attacks, and is referred to as the *NTT-protected implementation*.

Moreover, in this implementation, $\mathbf{y}$ is stored only in its NTT form $\hat{\mathbf{y}}$, reducing the memory overhead associated with retaining $\mathbf{y}$ in its normal form [11]. Compared to the reference

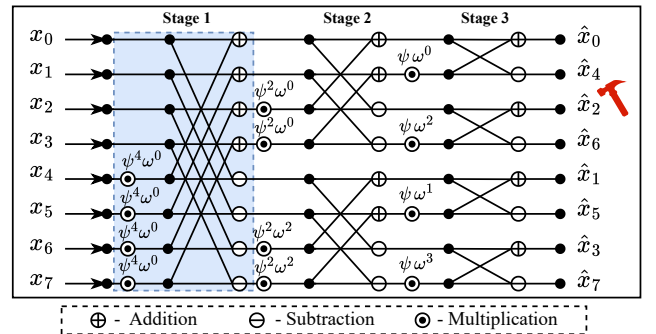


Fig. 1: NTT Operation on a Polynomial x with Degree 8

implementation, this design offers both memory efficiency and enhanced structural resilience against fault attacks. It is therefore a strong candidate for adoption at the implementation level by engineers. In this paper, we consistently refer to the implementation in (II.1) as the *protected implementation*.

D. Related Work

a) *Skip Addition Fault Attacks*: The skip addition fault attack was first introduced in [6] targeting lattice-based signature schemes. The authors described a fault model in which the attacker skips the addition operation such as $z = y + c \cdot s$. As a result, the masking polynomial y is omitted, thereby exposing the secret-dependent term $c \cdot s$ in the output signature z . While this work demonstrated the risk of secret leakage under such faults, it remained theoretical and did not involve actual fault injection. Building on this idea, a more practical and fine-grained fault model applied to coefficient-level additions in Dilithium was proposed in [7]. Each faulty signature leaks a single coefficient of $c \cdot s_1$, and by collecting $\ell \cdot n$ faulty signatures, the full s_1 can be recovered. This attack was further extended in [8], which analyzed its effectiveness in the randomized Dilithium. In particular, it was shown that if the fault causes $z'[i][j]$ to equal $y[i][j]$, the missing component $(c \cdot s_1)[i][j]$ can be recovered using signature correction techniques [15], thereby enabling recovery of s_1 .

b) *Loop-Abort Fault Attacks*: Loop-abort fault attacks form another class of threats. In [9], such an attack was first applied to Dilithium by aborting the sampling loop for the mask vector y , resulting in a large number of consecutive zero coefficients. Although the resulting signature remains valid, it leaks substantial information about the private key. The induced zeros in the high-degree coefficients of y effectively reduce its degree, enabling lattice reduction techniques to recover s_1 . However, this approach is vulnerable to shuffling countermeasures, which randomize the order of coefficients and prevent low-degree y even after loop abortion. Moreover, while the small norm of $c \cdot s_1$ enables partial inference of its coefficients from z , directly identifying equations like $z_{i,j} = (c \cdot s_1)_{i,j}$ by observing small-norm entries tends to produce noisy and unreliable systems. To address this, an ILP-based approach was proposed in [10], which tolerates noisy equations and still enables accurate recovery of s_1 .

c) *NTT-Specific Fault Attacks*: The first fault attack directly targeting the NTT operations in Dilithium was introduced in [11]. By injecting random faults into the twiddle pointers used in the NTT computation, the attacker probabilistically causes all twiddle factors to become zero. This attack was applied to the protected implementation described in Section II-C, enabling the attacker to obtain a large portion of the cs_1 vector and subsequently recover the private key s_1 .

III. ATTACK OVERVIEW

A. Adversary Model

Our attack adopts the *single-instruction skip* fault model, which is well studied and practical across various hardware platforms. It can be induced using standard techniques such

as clock glitching, voltage manipulation, electromagnetic interference, and laser injection.

We assume the existence of a physical device capable of performing multiple runs of the Dilithium signing routine. This device possesses an unknown private key but exposes its corresponding public key. The attacker is assumed to be capable of triggering a targeted fault during each signing routine and collecting the resulting faulty signatures.

We inject the fault during the first iteration of the internal rejection sampling loop. This eliminates timing uncertainty caused by the Fiat-Shamir with aborts structure [16]. To ensure that a faulty signature is output by the device, it must pass the rejection check in the first round. The probability of this happening for Dilithium-II, III, and V is approximately 0.22, 0.19, and 0.25, respectively. For clarity in theoretical analysis, we assume this condition always holds. Moreover, since all proposed attacks cause multiple coefficients in the sampled vector y to be zeroed, the actual success probabilities tend to be slightly higher in practice.

B. Attack Strategies

We propose three targeted categories of fault attack strategies, summarized as follows:

- **Strategy I: Skipping Polynomial-Level Addition.**

Skipping the critical function call in the vector addition for computing z disrupts the addition structure, causing a large number of additions to be skipped. As a result, polynomials within the secret vector $c \cdot s_1$ may become directly exposed in the faulty signature z' .

- **Strategy II: Loop-Abort in Vector Addition.**

This strategy targets the loop structure within the vector addition for computing z . By injecting a fault that prematurely aborts the loop, only a partial sum of coefficients is computed, exposing certain coefficients of the secret vector $c \cdot s_1$ in the resulting faulty signature z' .

- **Strategy III: Zeroing Twiddle Factors in NTT.**

This strategy targets the NTT computation of polynomials. By deliberately zeroing a carefully chosen minimal subset of twiddle factors used in the NTT, it causes more coefficients in the polynomial to become zero. Applied to NTT-protected implementations, this enables the leakage of $c \cdot s_1$ coefficients through the faulty signature z' .

All the proposed attacks in this work aim to recover the coefficients of the secret vector $c \cdot s_1$. For each leaked coefficient $(c \cdot s_1)[i][j]$, we define $\text{rot}(c, j) = \{c_j, c_{j-1}, \dots, c_0, -c_1, \dots, -c_{j+1}\}$, and construct a linear equation with the coefficients of s_1 as unknowns:

$$\langle \text{rot}(c, j), s_1[i] \rangle = (c \cdot s_1)[i][j]$$

By collecting $\ell \cdot n$ such equations, the private key s_1 can be recovered by solving the resulting system of linear equations.

The following subsections present concrete attacks based on these strategies and demonstrate how they can be effectively realized under the single-instruction skip fault model.

IV. CONCRETE FA METHODS

A. Function-Call Skipping Attacks

1) *Polyvec-Level Addition Skip Attack*: As shown in Listing 1, in all known implementations of Dilithium—including both the deterministic and randomized reference versions as well as the *pqm4* implementation—the computation of $\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}_1$ in the signing procedure follows a consistent pattern: the secret vector $c \cdot \mathbf{s}_1$ is first stored in $\mathbf{z}$, and then $\mathbf{z} = \mathbf{z} + \mathbf{y}$ is performed to compute the final result.

In this attack, skipping the function call on line 4 of Listing 1—which invokes the vector addition routine and corresponds to the `bl` instruction on line 6 in Listing 2—effectively eliminates the entire addition process. As a result, the faulted $\mathbf{z}'$ is guaranteed to pass the norm check at Line 15 of Algorithm II.2, and is subsequently output as the signature:

$$\mathbf{z}' = c \cdot \mathbf{s}_1$$

Fault injection is considered successful if all coefficients of the faulty signature $\mathbf{z}'$ have small magnitude, indicating that the addition of $\mathbf{y}$ was skipped. This is justified by the bounds $\|c \cdot \mathbf{s}_1\| < \beta$ and $\|\mathbf{y}\| < \eta$, where $\beta \ll \eta$. Further, the private key vector $\mathbf{s}_1$ can be recovered using the following relation:

$$\mathbf{s}_1 = \mathbf{z}' \cdot c^{-1}$$

Since the only difference between the reference and protected implementation lies in whether the operand $\mathbf{y}$ is represented in the NTT domain, both implementations perform the same vector addition operation. Therefore, this attack is effective against both implementations. With only a single faulty signature, it enables full recovery of the private key vector $\mathbf{s}_1$, thereby easily breaking the security of Dilithium.

2) *Poly-Level Addition Skip Attack*: As shown in Listing 3, the reference implementation of the `polyveccl_add` function performs vector addition by invoking the `poly_add` function ℓ times. The protected implementation adopts the same structure. Listing 4 presents the correspondi assembly code.

In both the reference and protected implementations, the ℓ polynomial additions invoked by `poly_add` are independent of each other. Therefore, skipping the `bl` instruction on line 3, which calls `poly_add`, effectively omits one polynomial addition without affecting the others in the vector.

Depending on the fault injection timing, it is possible to selectively skip any one of the ℓ polynomial additions in the

Listing 1: Official C Code Calling `polyveccl_add`

```
1 ...
2 polyveccl_pointwise_poly_montgomery(&z, &cp, &s1);
3 polyveccl_invntt_tomont(&z);
4 polyveccl_add(&z, &z, &y);
5 ...
```

Listing 2: Corresponding Assembly Code Snippet

```
1 ...
2 add.w    r1, sp, #11264
3 adds     r1, #32
4 mov      r0, r1
5 add.w    r2, sp, #7200
6 bl       <dilithium_polyveccl_add>
```

Listing 3: Official C Code for Polynomial Vector Addition

```
1 void polyveccl_add (polyveccl *w, const polyveccl *u,
2     const polyveccl *v) {
3     unsigned int i;
4     for (i = 0; i < L; ++i) {
5         poly_add(&w->vec[i],
6             &u->vec[i], &v->vec[i]);
7     }
8 }
```

Listing 4: Assembly Code Snippet for Polynomial Vector Addition

```
1 ...
2 add.w    r4, r4, #1024
3 bl       <dilithium_poly_add>
4 cmp.w    r4, #4096
5 bne.n    <dilithium_polyveccl_add+0xa>
```

vector addition process. Suppose the fault causes the k -th polynomial addition in the vector to be omitted. The resulting faulty output $\mathbf{z}'[i]$ can thus be uniformly described by (IV.1). In this context, S_1 represents the index set of polynomials unaffected by the fault, while S_2 contains the index of the faulty polynomial. For this case, we have $S_1 = \{0, 1, \dots, \ell-1\} \setminus \{k\}$ and $S_2 = \{k\}$. Therefore, $(c \cdot \mathbf{s}_1)[k]$ can be directly obtained by the attacker and used to construct linear equations for recovering $\mathbf{s}_1[k]$.

$$\mathbf{z}'[i] = \begin{cases} \mathbf{y}[i] + (c \cdot \mathbf{s}_1)[i], & i \in S_1 \\ (c \cdot \mathbf{s}_1)[i], & i \in S_2 \end{cases} \quad (\text{IV.1})$$

B. Loop-Abort Attacks in Addition Routines

Vector addition consists of element-wise operations and is typically implemented using loops. In Dilithium, the operands are polynomials over a ring, and polynomial addition is inherently coefficient-wise, also relying on loops. This makes the vector addition step a natural target for loop-abort fault attacks.

1) *Polynomial-Level Loop Abort*: As shown in Listing 3, the `polyveccl_add` function performs one polynomial addition per iteration. Suppose the loop is aborted during the $(k-1)$ -th iteration (0-based indexing), specifically by skipping the conditional branch instruction `bne.n` at line 5 in Listing 4. This prevents control from returning to the loop body and results in early termination. The resulting faulty output $\mathbf{z}'$ can be characterized by (IV.1). Under this attack model, the sets in that equation take the following values:

$$S_1 = \{0, 1, \dots, k-1\}, \quad S_2 = \{k, k+1, \dots, \ell-1\}, \quad k \geq 1$$

Depending on the point at which the loop aborts, all polynomials of the secret vector $c \cdot \mathbf{s}_1$ —except the first one—may be exposed in the faulty signature $\mathbf{z}'$. Notably, this attack typically fails to recover the first polynomial of the private key vector $\mathbf{s}_1$. However, when polynomial-level shuffling is applied as a countermeasure, the randomized polynomial addition order may lead to the first polynomial being leaked in the faulty signature. In such cases, only two faulty signatures are sufficient to recover the full $\mathbf{s}_1$. This behavior is consistent across both deterministic and randomized Dilithium and applies to both the reference and protected implementations.

2) *Coefficient-Level Loop Abort*: As shown in Listing 5, the `poly_add` function implements the addition of two polynomials of degree $n-1$ by looping to perform coefficient-wise addition. Suppose a fault is injected into the i -th polynomial component of the vector, and the addition loop is aborted after the $(k-1)$ -th coefficient has been processed. This is achieved by skipping the `bne.n` instruction on line 8 in Listing 6 within the corresponding loop. For the reference implementation, the resulting faulty output $\mathbf{z}'[i]$ can be expressed as:

$$\mathbf{z}'[i][j] = \begin{cases} \mathbf{y}[i][j] + (c_{s_1})[i][j], & j \in \{0, \dots, k-1\} \\ (c_{s_1})[i][j], & j \in \{k, \dots, n-1\} \end{cases} \quad (\text{IV.2})$$

Again, the number of leaked coefficients in $c \cdot s_1$ depends on the abort point k . Since $y[i][j]$ is drawn uniformly from $[-\gamma_1, \gamma_1]$ and $(c_{s_1})[i][j] \approx \mathcal{N}(0, \sigma^2)$ with $\gamma_1 \gg \beta > \sigma$, we have $\Pr(|z[i][j]| \leq \beta) \lesssim \beta/\gamma_1 \leq 1.50 \times 10^{-3}$. Hence, even if k is unknown, the faulty output $\mathbf{z}'$ exhibits a contiguous run of small-norm coefficients ($< \beta$) for $j \geq k$, which allows us to localize k and extract a linear system in c_{s_1} .

Only two faulty signatures are required to recover any polynomial of s_1 when $k \leq n/2$. When k attains its minimum value (i.e., $k=1$), only $(c_{s_1})[i][0]$ remains unknown; since the coefficients are approximately Gaussian and $\|c \cdot s_1\| \leq \beta$, we enumerate $\{0, \pm 1, \pm 2, \dots\}$ outward from the mean and solve the resulting linear system. We then validate the candidate by checking whether the recovered s_1 satisfies $\|s_1\|_\infty \leq \eta$ (with $\eta \in \{2, 4\}$); incorrect guesses typically yield solutions whose ℓ_∞ -norm far exceeds η . To accommodate imprecise loop-abort faults (which may introduce a few noisy equations) or very small k , an Integer Linear Programming (ILP) formulation can also recover one polynomial of s_1 from a *single* faulty signature, following [10].

C. Single-Twiddle-Factor Zeroing Attack

In the first stage of the Cooley–Tukey NTT (CT-NTT) [17] implementation, all butterfly operations belong to the same butterfly group and therefore use a shared twiddle factor, which in this group is given by $\psi^4 \cdot \omega^0$, as shown in Fig. 1. Consequently, faulting this single twiddle factor affects all $n/2$ butterfly operations within the stage. Moreover, the *normal-domain* coefficients of the polynomial directly participate in the first-stage computation, making the fault effects more predictable and easier to control.

Listing 5: Official C Code of Polynomial Addition

```
1 void poly_add (poly *c, const poly *a, const poly *b) {
2     unsigned int i;
3     for(i = 0; i < N; i++){
4         c->coeffs[i] = a->coeffs[i] + b->coeffs[i];}
```

Listing 6: Assembly Code Snippet of Polynomial Addition

```
1 ...
2 ldr.w r4, [r1, r3, lsl #2]
3 ldr.w r5, [r2, r3, lsl #2]
4 add r4, r5
5 str.w r4, [r0, r3, lsl #2]
6 adds r3, #1
7 cmp.w r3, #256
8 bne.n <dilithium_poly_add+0x4>
```

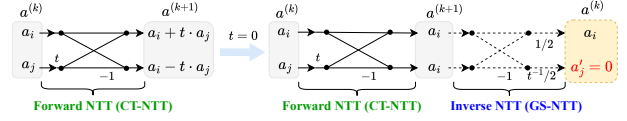


Fig. 2: Illustration of Twiddle Factor Zeroing in the Butterfly Operation

Let $a^{(k)}$ denote the output after the k -th stage of the CT-NTT. Assuming that the shared twiddle factor of a butterfly group in the k -th stage is zero, all butterfly operations within that group produce identical outputs at both branches; that is, for each butterfly pair (i, j) , the two output branches satisfy $a^{(k+1)}[i] = a^{(k+1)}[j]$. This is functionally equivalent to replacing the a_j input branch of each butterfly with zero, as illustrated in Fig. 2.

Based on this observation, we propose a novel fault model in which the twiddle factor used in the first-stage butterfly group is faulted to zero. Under this model, the first-stage output satisfies:

$$a^{(2)}[j] = a^{(2)}\left[j + \frac{n}{2}\right], \quad \text{for } j \in \left[0, \frac{n}{2} - 1\right], \quad n = 256$$

which effectively zeroes out the coefficients in degrees $n/2$ through $n-1$ of the polynomial that has undergone a faulted NTT computation. We refer to this as the *single-twiddle-factor zeroing fault model*. Notably, we do not constrain the specific fault injection technique used to achieve this zeroing effect; the proposed fault model is designed to remain independent of any particular implementation.

Nevertheless, we propose an implementation-aware strategy that achieves the desired zeroing effect by skipping a single instruction. Specifically, as illustrated in Listing 7, we skip the assignment to `zeta` at line 5, without affecting the increment of `k`, thereby causing `zeta` to retain its original memory value instead of being assigned the expected twiddle factor.

In our key-recovery attack model, we assume that the uninitialized `zeta` variable is read as zero—an assumption widely adopted in the fault-attack literature. For example, in loop-abort attacks [9], the unsampled coefficients of $\mathbf{y}$ are typically treated as zeros. Recent work [18] carried out several attacks based on the same assumption. In practice, uninitialized memory tends to contain zero with non-negligible probability. For instance, [11] shows that, in 10,000 randomized memory access tests, a 1KB memory region entirely filled with zeros appeared with a probability of 20–25%. Furthermore, zero-initialization is both a safe and widely adopted software practice, lending credibility to our model assumption.

Following this assumption, the single-twiddle-factor zeroing fault is injected during the NTT computation of $\mathbf{y}$, as shown in line 8 of Algorithm II.2. This fault corrupts the i -th polynomial of the vector $\mathbf{y}$ as follows:

$$\mathbf{y}'[i] = (y_0, y_1, \dots, y_{127}, 0, 0, \dots, 0), \quad i \in \{0, \dots, \ell-1\}.$$

Under the protected implementation described by (II.1), where the computation of the vector $\mathbf{z}$ is performed entirely in the NTT domain, the process becomes directly dependent

Listing 7: Official C Source Code of NTT

```

1 void ntt(int32_t a[N]){
2     ...
3     for(len=128; len>0; len>=1){
4         for(start = 0; start < N; start = j+len){
5             zeta = zetas[++k];
6             for(j = start; j < start + len; ++j){
7                 t=montgomery_reduce((int64_t)zeta*a[j+len])
8                 ;
9                 a[j+len]=a[j]-t;
10                a[j]=a[j]+t; }
11     }
12 }

```

on the faulted NTT -domain representation $\hat{y}'$. The resulting faulty signature z' can be described by (IV.2), where $k = 128$ denotes the number of unaffected coefficients.

As a result, half of the coefficients of $(c \cdot s_1)[i]$ are directly exposed in the faulty polynomial $z'[i]$ within the faulty signature. Consequently, only 2ℓ faulty signatures are sufficient to recover $\ell \cdot n$ independent linear equations, which is enough to reconstruct the full private key vector s_1 .

V. EXPERIMENT

Our experiment is divided into two main phases: the fault injection phase and the private key recovery phase.

A. Experimental Setup

1) *Fault Injection Phase*: The fault injection phase is conducted on the ChipWhisperer-Lite platform, as illustrated in Fig. 3. We take the reference implementation of Dilithium-2, as well as a corresponding protected implementation, which is obtained by making minor modifications to directly perform additions on the NTT -domain representations of polynomials, as a case study, where the signing program is compiled using `arm-none-eabi-gcc` with the flags `-Os`, `-mcpu=cortex-m4`, `-mfloat-abi=soft`, and `-DF_CPU=7372800`, and executed on an ARM Cortex-M4 target board equipped with an STM32F405 microcontroller. We employ *clock glitching* as the fault injection technique to precisely skip individual instructions during execution. A trigger flag is inserted immediately before and after the vector addition operation shown in Listing 1, line 4, to set the fault injection window and synchronize the injection timing.

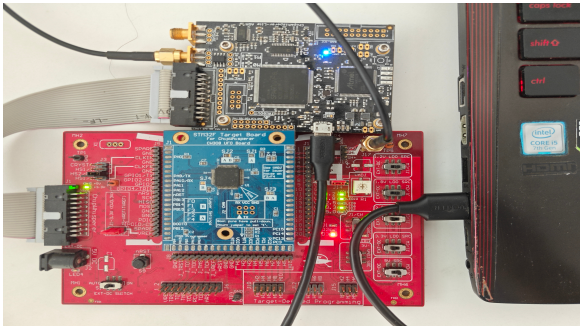


Fig. 3: Chipwhisperer-Lite Platform

TABLE II: Fault Injection Parameters for the Proposed Attacks

Attack	Offset	Width	Ext_Offset	Success Rate
<i>Polyveccl-Addition Skip</i>	-48.75	-3.25	7	57%
<i>Poly-Addition Skip</i>	-47.75	-1.5	13	40%
<i>Poly-Level Loop Abort</i>			2843	29%
<i>Coeff-Level Loop Abort</i>	-49	-3.25	$34 + 11k$	34%
<i>Twiddle Factor Zeroing</i>	-47.75	-1.5	19	63%

The host PC communicates with the DUT via UART. A Python script running on the host coordinates trigger generation, glitch parameter tuning, and data collection.

2) *Private Key Recovery Phase*: In this phase, the coefficients of $c \cdot s_1$ extracted from the collected faulty signatures are used to construct a linear system, which is then solved using the Z3 solver to recover the private key s_1 . The recovery is performed on a host machine equipped with an Intel Core i7-12700 CPU running Ubuntu 20.04.

B. Experimental Results

The fault injection framework provided by the ChipWhisperer-Lite platform supports three key parameters for precise clock glitch injection: `offset`, `width`, and `ext_offset`. The `offset` parameter defines the temporal shift of the glitch within the target clock cycle. The `width` specifies the width of the glitch cycle. The `ext_offset` determines the number of clock cycles between the trigger signal and the target instruction, allowing synchronization of the injection point with a specific operation.

These parameters are crucial for accurately targeting and skipping specific instructions during execution. Table II presents the results of fault injection experiments conducted on signature operations over 1,000 randomly generated messages. It summarizes the glitch parameters used in all our proposed attacks, along with their corresponding success rates. Notably, the results from our practical experiments on *function-call skipping* and *loop-abort* attacks show that both the reference and protected implementations share identical clock glitch parameters and exhibit very similar success rates. This can be naturally explained by the fact that the target operations requiring faults are exactly the same in both implementations under these attacks. The `ext_offset` values in the table represent the parameters used during the addition of the first polynomial in the vector, where $k \in \{0, 255\}$ denotes the index of the coefficient-wise addition.

In the single-twiddle-factor zeroing attack, a minor modification was made by zero-initializing the memory region storing `zeta`, similar to the practice in [8], which instead zero-initializes the memory region of $\hat{A}$ in the signing process. The code was compiled with the `-O0` flag to disable compiler optimizations, while all other compilation settings remained unchanged. A trigger flag was inserted before and after line 5 of Listing 7. Under precisely tuned fault injection parameters (as shown in Table II), a total of 567 clock glitches were

injected. In 358 of these attempts, the instruction that stores the twiddle factor into `zeta` was successfully skipped, yielding a fault success rate of 63.2%.

VI. BYPASSING SHUFFLING COUNTERMEASURES

For attacks mounted at the polynomial level, the norm of the term $c \cdot s_1$ is bounded by β (with $\beta \leq 196$), and thus the faulty polynomial $\mathbf{z}$ —which consists of n small-norm coefficients—can be identified simply by inspecting its coefficients. This allows the attacker to locate the target polynomial affected by the skip fault and subsequently construct a system of linear equations for private key recovery. As a result, both coefficient-level and polynomial-level shuffling countermeasures are ineffective against attacks such as the polyvec-level addition attack, the poly-level addition attack, and the polynomial-level loop abort attack.

The effect of the coefficient-level loop abort attack is analogous to that of the loop-abort attack targeting the generation of $\mathbf{y}$ -coefficients in [9]. Although that attack is susceptible to coefficient-level shuffling, which was proposed as an effective countermeasure in [9], this defense strategy has been successfully bypassed in [10] using integer linear programming (ILP)-based methods. Therefore, such attacks remain effective and can successfully bypass shuffling countermeasures.

As for the single-twiddle-factor zeroing attack, it targets the first stage of the NTT, which contains only a single butterfly group. Since there is no inter-group shuffling at this stage, and intra-group shuffling does not affect the outcome—because all the faulted coefficients are zeroed out regardless of the butterfly operation order—making shuffling at any granularity [19] ineffective against this type of attack.

In summary, all the fault attacks proposed in this work can effectively bypass shuffling countermeasures.

VII. CONCLUSION

In this work, we systematically investigate skip addition-based fault attacks and extend the attack granularity from the coefficient level to the polynomial level. This transition significantly reduces the number of required faulty signatures and improves the efficiency and practicality of private key recovery. We further explore the fault vulnerability of storage-optimized protected implementation and propose a novel fault model targeting the Number Theoretic Transform, which effectively zeroes out multiple coefficients in a polynomial. Our experimental results demonstrate that the proposed attacks achieve full private key recovery across all security levels and various Dilithium implementations, highlighting the broad applicability and practical relevance of our approach.

ACKNOWLEDGMENT

This work is supported in part by the National Natural Science Foundation of China (No. U2336205, No. 62202230, No. 62202231, No. 62302224, No. 62302226).

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 124–134, 1994.
- [2] National Institute of Standards and Technology (NIST), "FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)," U.S. Department of Commerce, Aug. 13, 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.204>
- [3] P. Ravi, A. Chattopadhyay, J.-P. D'Anvers, and A. Baksı, "Side-channel and fault-injection attacks over lattice-based post-quantum schemes (Kyber, Dilithium): Survey and new results," *ACM Trans. Embedded Comput. Syst.*, vol. 23, no. 2, pp. 35:1–35:54, 2024.
- [4] M. Randolph and W. Diehl, "Power side-channel attack analysis: A review of 20 years of study for the layman," *Cryptography*, vol. 4, no. 2, p. 15, 2020.
- [5] A. Barengi, L. Breveglieri, I. Koren, and D. Naccache, "Fault injection attacks on cryptographic devices: Theory, practice, and countermeasures," *Proceedings of the IEEE*, vol. 100, no. 11, pp. 3056–3076, Nov. 2012.
- [6] N. Bindel, J. Buchmann, and J. Krämer, "Lattice-based signature schemes and their sensitivity to fault attacks," in *Proc. Fault Diagnosis and Tolerance in Cryptography (FDTC)*, 2016, pp. 63–77.
- [7] P. Ravi, M. P. Jhanwar, J. Howe, A. Chattopadhyay, and S. Bhasin, "Exploiting determinism in lattice-based signatures: Practical fault attacks on pqm4 implementations of NIST candidates," in *Proc. ACM Asia Conf. Comput. Commun. Secur. (AsiaCCS)*, Auckland, New Zealand, Jul. 2019, pp. 427–440.
- [8] E. Krahmer, P. Pessl, G. Land, and T. Güneysu, "Correction fault attacks on randomized CRYSTALS-Dilithium," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 3, pp. 174–199, 2024.
- [9] T. Espitau, P.-A. Fouque, B. Gérard, and M. Tibouchi, "Loop-abort faults on lattice-based Fiat-Shamir and Hash-and-Sign signatures," in *Proc. Sel. Areas Cryptogr. (SAC)*, vol. 10532, St. John's, NL, Canada, Aug. 2016, pp. 140–158.
- [10] V. Q. Ulitzsch, S. Marzougui, A. Bagia, M. Tibouchi, and J.-P. Seifert, "Loop aborts strike back: Defeating fault countermeasures in lattice signatures with ILP," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2023, no. 4, pp. 367–392, 2023.
- [11] P. Ravi, B. Yang, S. Bhasin, F. Zhang, and A. Chattopadhyay, "Fiddling the twiddle constants: Fault injection analysis of the number theoretic transform," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2023, no. 2, pp. 447–481, 2023.
- [12] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. Schanck, P. Schwabe, M. Seiler, and D. Stehlé, "Reference implementation of Dilithium," GitHub repository, [Online]. Available: <https://github.com/pq-crystals/dilithium>, accessed Nov. 1, 2024.
- [13] M. J. Kannwischer, R. Petri, J. Rijneveld, P. Schwabe, and K. Stofelen, "PQM4: Post-quantum crypto library for the ARM Cortex-M4," [Online]. Available: <https://github.com/mupq/pqm4>, 2020.
- [14] L. Groot Bruinderink and P. Pessl, "Differential fault attacks on deterministic lattice signatures," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 3, pp. 21–43, 2018.
- [15] S. Islam, K. Mus, R. Singh, P. Schaumont, and B. Sunar, "Signature correction attack on Dilithium signature scheme," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2022, pp. 647–663.
- [16] V. Lyubashevsky, "Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures," in *Advances in Cryptology – ASIACRYPT 2009*, vol. 5912, M. Matsui, Ed. Berlin, Heidelberg: Springer, 2009, pp. 598–616.
- [17] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [18] S. Amer, Y. Wang, H. Kippen, T. Dang, D. Genkin, A. Kwong, A. Nelson, and A. Yerukhimovich, "PQ-Hammer: End-to-End Key Recovery Attacks on Post-Quantum Cryptography Using Rowhammer," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2025, pp. 3567–3582.
- [19] P. Ravi, R. Poussier, S. Bhasin, and A. Chattopadhyay, "On configurable SCA countermeasures against single trace attacks for the NTT – a performance evaluation study over Kyber and Dilithium on the ARM Cortex-M4," in *Proc. 10th Int. Conf. Security, Privacy and Applied Cryptography Engineering (SPACE)*, 2020, pp. 123–146.